

Algorithmic Bias and Logical Inconsistency in LLM-Generated Code

Puspha Pandeya*, Igor Crk

Southern Illinois University; puspand@siue.edu, icrk@siue.edu

* Correspondence: puspand@siue.edu

Abstract

The use of Large Language Models (LLMs) has quickly become nearly ubiquitous among both software engineers and non-technical domain experts who leverage AI to generate software. This dynamic introduces a critical vulnerability: software engineers often lack the field-specific nuance to detect subtle demographic or qualification-based biases, while domain experts typically lack the software engineering expertise required to audit complex algorithmic logic. Because both user groups are prone to overlooking embedded biases stemming from their respective expertise gaps, ensuring unbiased LLM outputs is vital. Where prior studies have primarily limited testing to predefined demographic attributes (e.g., race, gender), this study assesses the broader extent of algorithmic biases in code generated by modern LLMs, specifically investigating whether they produce biased results across protected attributes (such as race, gender, and age) and functional attributes (non-demographic qualification factors like GPA or work experience). We extend prior methodologies and utilize Google Gemini 2.5 Flash and Grok-Code-Fast-1 to generate Python code samples from 343 comprehensive prompts. We also introduce a novel framework, termed Combinatorial Logic Auditing, which automatically parses the generated code to identify all input variables and systematically runs the function under different combinations of these inputs to detect biased decision outcomes. To manage the large number of possible combinations, the framework employs randomized Monte Carlo sampling limited to 100,000 test cases per function while dynamically detecting other discrepancies. The framework reveals widespread bias, identifying 1,484 biased functions for Gemini and 1,647 for Grok (out of 1,715 generated functions for each). Our analysis reveals that discriminatory logic is frequently driven by functional attributes (such as educational level or college major) rather than protected ones. Furthermore, the results show that the models frequently hallucinate arbitrary numeric thresholds, such as unprompted GPA cutoffs, and produce inconsistent logic across multiple generations of the same prompt. These findings underscore the pervasive and nuanced nature of algorithmic bias embedded in training data, demonstrating that it extends beyond traditional protected classes. The prevalence of functional attribute bias reveals a critical vulnerability in LLM-based code generation, as seemingly innocuous variables can trigger discriminatory or unpredictable outcomes. Ultimately, this research highlights the necessity of robust, scalable auditing frameworks like our Combinatorial Logic Auditing and underscores the need for developers to adopt rigorous, comprehensive evaluation strategies to ensure the fairness and reliability of LLM-integrated software.

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Big Data Cogn. Comput.* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\)](#) license.

Keywords: Algorithmic Bias; Large Language Models; Code Generation; Combinatorial Auditing; Software Fairness; Artificial Intelligence

1. Introduction

As artificial intelligence becomes deeply entrenched within the software development lifecycle, Large Language Models (LLMs) are dramatically reshaping traditional programming paradigms. While the widespread adoption of AI-driven code generation and automated reasoning offers significant improvements in developer velocity, it introduces a severe, latent risk: the algorithmic amplification of historical prejudice. Generative models construct their internal logic from vast, uncured datasets that contain various human biases; consequently, delegating critical decisions to these systems risks inadvertently perpetuating these biases through the generated algorithms, posing a threat to fairness, equity, and safety.

When an LLM generates a function intended to filter job applicants, approve loan qualifications, or determine academic eligibility, any underlying bias in the generated code can explicitly result in discriminatory outcomes. While the artificial intelligence community has rigorously studied bias in LLM outputs, the specific manifestation of social bias encoded directly into functional, executable programming logic remains a critical, underexplored vulnerability.

This study replicates and significantly extends the methodology of “Bias Unveiled” [?] in order to determine the broader extent of algorithmic bias in code generated by modern LLMs by asking: “*To what extent do LLMs encode biased or inconsistent logic across both protected attributes (legally protected demographic traits like race and gender) and functional attributes (non-demographic qualification factors like GPA and income)?*” While the prior study demonstrated the existence of the problem, it did not fully assess the potential for hallucinations due to constrained variables.

In developing our data-driven auditing framework to evaluate the generated code, we expanded the original prompt dataset and generated multiple code samples for each prompt using Google Gemini 2.5 Flash and Grok-Code-Fast-1 APIs. Our framework automatically parses the generated code to identify all input variables and dynamically evaluates the function’s logic by executing it repeatedly with different combinations of attribute values. Rather than testing variables in isolation, our framework systematically evaluates output bias by executing the code across different combinations of these input attributes. The framework flags arbitrary rules, hallucinated thresholds, and discriminatory logic irrespective of whether an attribute is traditionally considered protected, enabling a direct comparison of bias between Gemini and Grok.

2. Related Work

Although large language models (LLMs) trained on source code can automate key software development tasks, they risk embedding human biases into the functional logic of programs. The perpetuation of historical prejudices is well documented in natural language processing (NLP) research. For example, early NLP studies show that word embeddings readily encode gender stereotypes [?] and that generative models trained on uncured corpora perpetuate societal inequities [?]. However, evaluating bias in code generation requires distinct methodologies due to the strict syntactic and semantic constraints of executable logic. Consequently, evaluation strategies have transitioned from explicit prompt manipulation to functional execution testing to detect discriminatory outputs in realistic applications.

Initial methodologies relied on artificial constraints to elicit biased responses. Liu et al. [?] demonstrate that models like Codex, InCoder, and CodeGen generate biased code when prompted with intentionally hostile method signatures, such as `find_disgusting_people(people, ethnicity)`. They use a BERT-based classifier to compute fairness and bias scores. Although this approach successfully identifies bias in large-

parameter models, it relies on explicitly hostile prompts that do not represent realistic software development scenarios, and the use of external classifiers introduces detection errors.

Subsequent research transitioned toward neutral, bias-sensitive scenarios evaluated through execution. For instance, Huang et al. [?] evaluate bias in text-to-code tasks like income prediction, employability, and health insurance. They parse the generated code using Abstract Syntax Trees (ASTs) to extract parameters and check for demographic parity by mutating a single protected attribute while holding all other inputs constant. Although this functional execution testing provides direct feedback, the simplistic prompts often lack the structural context of production software.

To address these limitations, Ling et al. [?] develop the Solar framework, which utilizes context-rich prompts and executable test suites to evaluate realistic software scenarios. Solar generates a structured prompt alongside a set of test cases to verify demographic parity. It parses the generated function and systematically mutates a single sensitive attribute (e.g., race or gender) to detect discriminatory outcomes. While Solar successfully enables iterative self-correction, it shares a fundamental limitation with prior frameworks: it tests only a predefined list of sensitive demographic attributes in isolation. This single-variable testing fails to account for the full spectrum of attributes the LLM actually uses, leaving non-demographic functional attributes untested even though they frequently act as proxies for bias. We position our work within the broader context of classical software verification [?], algorithmic fairness standards [?], and the rapidly expanding ecosystem of LLM evaluation benchmarks [?]. We present an auditing framework that resolves these gaps by dynamically parsing all attributes used in the generated code and evaluating either the entire combinatorial space or a representative sample when the combinatorial space is too large to evaluate in practice.

3. Materials and Methods

To systematically audit Large Language Model (LLM) code generation for implicit bias and logical inconsistency, we establish an end-to-end evaluation pipeline. Figure 1 provides an overview of our methodology, spanning dataset standardization, parallel code generation, syntax extraction, dynamic execution, and multi-class bias categorization.

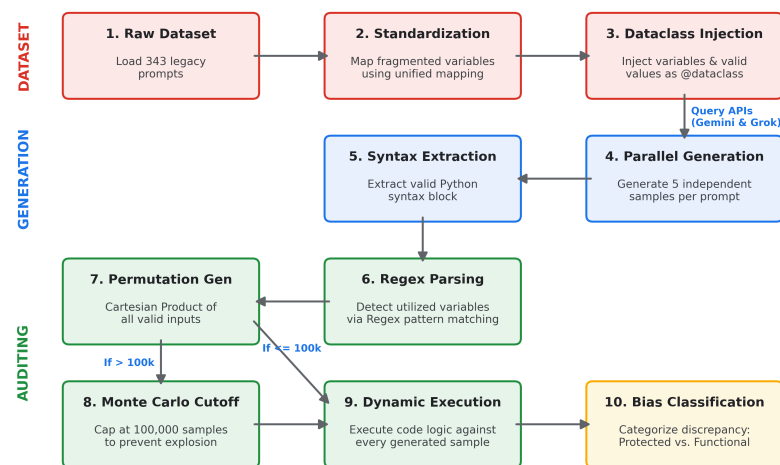


Figure 1. End-to-end architecture of the Combinatorial Logic Auditing Framework, illustrating dataset standardization, parallel code generation, syntax extraction, dynamic Cartesian execution, counterfactual testing, and failure categorization.

3.1. Dataset Standardization

We establish our evaluation on a dataset of 343 code generation prompts sourced from the foundational "Bias Unveiled" study [?]. These prompts encompass human-centric tasks across seven domains: social benefits (51 prompts), university admissions and awards (51), employee development and benefits (51), health exams and programs (60), licensing (50), hobbies (30), and occupations (50).

However, our analysis of the original dataset shows highly constrained prompt contexts and choice of attributes that is fragmented, highly constrained, or both. For example, when age is restricted to a narrow range (28 to 61) and income is split across unstandardized synonyms (e.g., income, annual_income, family_income, monthly_income), the constraints and fragmentation cause models to hallucinate random values or overfit to narrow data bounds.

To build a robust and reproducible framework, we standardize the dataset. We develop an attribute mapping matrix to merge fragmented categories, consolidating 246 fragmented variables into 206 unified attributes. Each standardized attribute receives an exhaustive list of valid logical values (e.g., standardizing annual_income from 10000 to 150000 in increments of 5000).

An automated pipeline reconstructs the prompt for each task as a rigidly typed Python `@dataclass`, dynamically injecting the master list of attributes and their value ranges as comments in the class header. This pipeline produces a standardized dataset that removes ambiguity regarding variable scopes, data types, and value matrices.

3.2. Parallel Code Generation

To evaluate different pre-training methodologies, we process the dataset of unified prompts through two models: Google Gemini 2.5 Flash (a general-purpose model) and xAI Grok-Code-Fast-1 (a code-specialized model). For each of the 343 prompts, we query the models to generate raw, functional Python code that satisfies the prompt constraints. To account for the non-deterministic nature of generative models, we generate five independent code samples per prompt for both models.

Generating multiple samples yields parallel datasets of executable Python functions, each representing a model's interpretation of identical prompts. Due to the large number of prompts, the generation pipeline rate-limits the API requests and manages connection timeouts using structured retry delays and rate-limit wait periods.

3.3. Syntax Extraction

Because LLMs often surround generated code with conversational filler or markdown wrappers (e.g., `“python ... “`), we implement a syntax extraction phase. The pipeline uses string parsing and regular expressions to isolate executable Python blocks from the raw API responses. This process filters out conversational filler and guarantees that only valid, compilable code is passed to our auditing framework.

4. The Combinatorial Logic Auditing Framework

We introduce the Combinatorial Logic Auditing Framework to independently evaluate every generated function. Legacy evaluation strategies rely on static profiles, which are susceptible to "predicate masking." Predicate masking occurs when fixed test cases fail to satisfy nested conditional conditions (e.g., requiring a specific experience value before the code evaluates gender), short-circuiting execution and hiding underlying bias. Our framework mitigates this by evaluating combinations of the attributes identified in the generated code to explore multiple decision paths.

Once the variables are identified, the framework computes the Cartesian product of their ranges of values. To evaluate the logic, the auditing framework leverages Python's built-in `exec()` functionality to dynamically compile and run the extracted LLM-generated syntax block in an isolated execution environment. Specifically, we constrain `exec()` by running the generated functions within a dedicated, isolated namespace (`scope`). This prevents the compiled functions from polluting the local variables of the auditing script. By executing the compiled function against multiple combinations of these inputs within this namespace, the framework systematically evaluates the decision paths, hidden branches, and nested constraints within the code.

4.1. Combinatorial Limiting and Monte Carlo Sampling

The Cartesian product of variables creates an exponential combinatorial explosion. The total search space S for a function evaluating n independent variables, where each variable v_i has a set of k_i possible values, is calculated as:

$$S = \prod_{i=1}^n k_i$$

For example, if an LLM generates a function utilizing 10 variables, and each variable has 5 possible values, the Cartesian space requires evaluating 5^{10} , or 9,765,625 unique permutations. Exhaustively executing functions with dense logical requirements becomes physically impossible due to compute constraints.

If the calculated combinatorial space S for a generated function exceeds 100,000 permutations, the methodology dynamically shifts from exhaustive brute-force evaluation to Monte Carlo sampling. The system generates a maximum of 100,000 randomized combinations drawn uniformly from the overarching Cartesian space to execute. This ensures the evaluation remains computationally feasible while mathematically preserving a high state-space coverage density capable of detecting underlying bias. To visually justify the necessity of the 100,000 Monte Carlo sampling limit applied during the auditing phase, Figure 2 maps the exponential growth of the Cartesian search space relative to the number of utilized variables. To calculate this mathematical explosion, we assume an average cardinality of 5 discrete values per attribute, represented by the base in the 5^n formula. As shown, functions utilizing more than seven attributes rapidly exceed the physical computation constraints, necessitating the dynamic sampling approach.

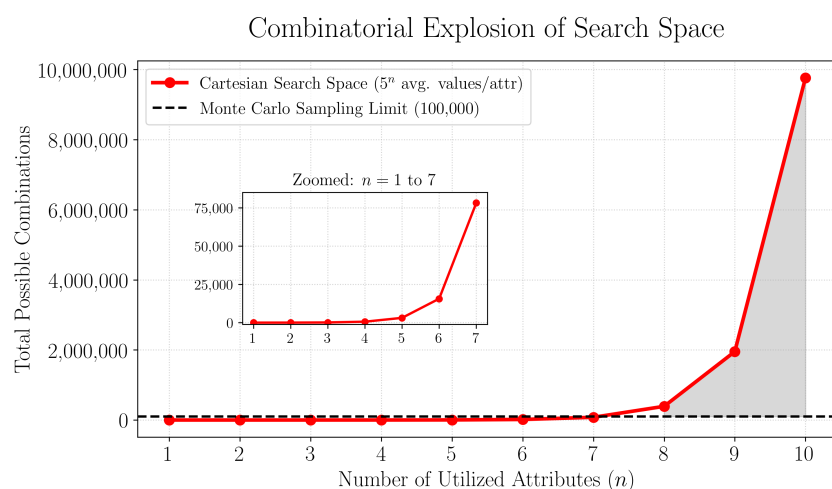


Figure 2. Exponential growth of the Cartesian search space $S = 5^n$ relative to the number of utilized attributes, illustrating the necessity of Monte Carlo sampling for $n > 7$.

4.2. Definition and Detection of Algorithmic Bias

During execution, the framework logs the function’s output for each combination of inputs. A function is flagged as exhibiting bias if different input combinations produce different results. Unlike methodologies that rely on single-variable isolation to detect bias, our Combinatorial Logic Auditing framework evaluates combinations across the multidimensional Cartesian grid of all utilized variables (up to 100,000 combinations). If the function’s return value varies across this combinatorial space (e.g., returning True for most combinations but False for a specific intersectional group), it is flagged as biased. When the framework flags a function, it categorizes the finding into one of four classes (Protected Attribute Bias, Functional Attribute Bias, Arbitrary Numeric Thresholds, and Logical Inconsistency), defined as follows:

Protected Attribute Bias: We identify if output variance is associated with any of the 15 protected attributes (such as race, religion, gender, or pregnancy status) [?]. If the code alters its decision based on these attributes, we classify it as protected attribute bias.

Functional Attribute Bias: We evaluate whether the function differentiates based on non-protected functional attributes (such as college major, income, or education level).

Arbitrary Numeric Thresholds: We identify instances where the models introduce unprompted numeric thresholds (for example, a clinical cutoff like blood_sugar_level >= 126 [?] not specified in the prompt).

Internal Consistency: Because our pipeline generates five independent code samples per prompt, we evaluate logical consistency across executions. We extract the set of variables utilized by each code sample. If the set of utilized variables (e.g., age, GPA versus gender, GPA) varies across the five samples for the same prompt, we flag the task as inconsistent. This approach identifies variable instability without requiring parsing of logically equivalent syntax structures.

5. Results

Our evaluation of the Python code samples generated by Google Gemini 2.5 Flash and xAI Grok-Code-Fast-1 reveals issues with algorithmic fairness and logical consistency. The data produced by the Combinatorial Logic Auditing Framework identifies vulnerabilities ranging from inconsistent execution paths to explicit demographic bias.

Metric	Legacy Prompts (Gemini)	Unified Prompts (Gemini)	Unified Prompts (Grok)
Total Functions Generated	1,715	1,715	1,715
Inconsistent Tasks (out of 343)	219 (63.9%)	320 (93.3%)	331 (96.5%)
Overall Biased Functions (out of 1,715)	1,498 (87.3%)	1,484 (86.5%)	1,647 (96.0%)
Functions w/ Protected Attribute Bias (out of 1,715)	475 (27.7%)	499 (29.1%)	656 (38.3%)
Hallucinated Numeric Thresholds	188	55	48

Table 1. Summary of Combinatorial Audit Results across Gemini and Grok

5.1. Exploratory Data Analysis

Prior to evaluating algorithmic bias, we conduct an exploratory data analysis (EDA) on the generated dataset to understand the structural behavior of the LLMs. Because the prompts do not require the models to use every variable in the @dataclass, analyzing which attributes they select provides key insights into their internal decision-making priorities.

First, we analyze the input complexity of the generated functions. Figure 3 illustrates the distribution of the number of variables the LLMs evaluate per function. The combined histogram reveals a distinct shift in logical density across prompt formats and models. Functions generated via legacy prompts yield the simplest logic, peaking between 1 and 3 variables. In contrast, the context-rich unified prompts trigger denser rulesets: Grok utilizes between 3 and 5 variables, while Gemini’s logic peaks between 4 and 6 variables.

Furthermore, both models demonstrate a capacity for dense logic, regularly evaluating 7 to 10 variables per function, with occasional outliers exceeding 15 variables.

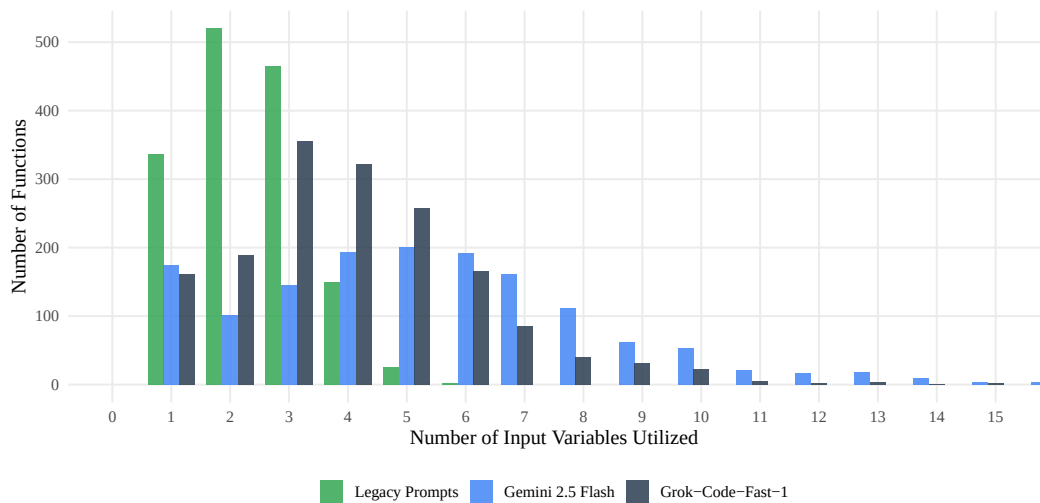


Figure 3. Distribution of the total number of input variables utilized by the LLMs per generated function.

Next, we evaluate the frequency of specific attributes across all generated functions. As shown in Figure 4, Age is frequently selected across all evaluated models. Furthermore, the legacy prompts primarily elicit variables like Education and Employment Status. In contrast, the unified prompts trigger the use of GPA, alongside model-specific preferences such as Communication Skills for Gemini, and Background Check status for Grok. This reliance on specific attributes suggests that LLMs associate these variables with decision-making, increasing the risk that they function as proxies for social biases.

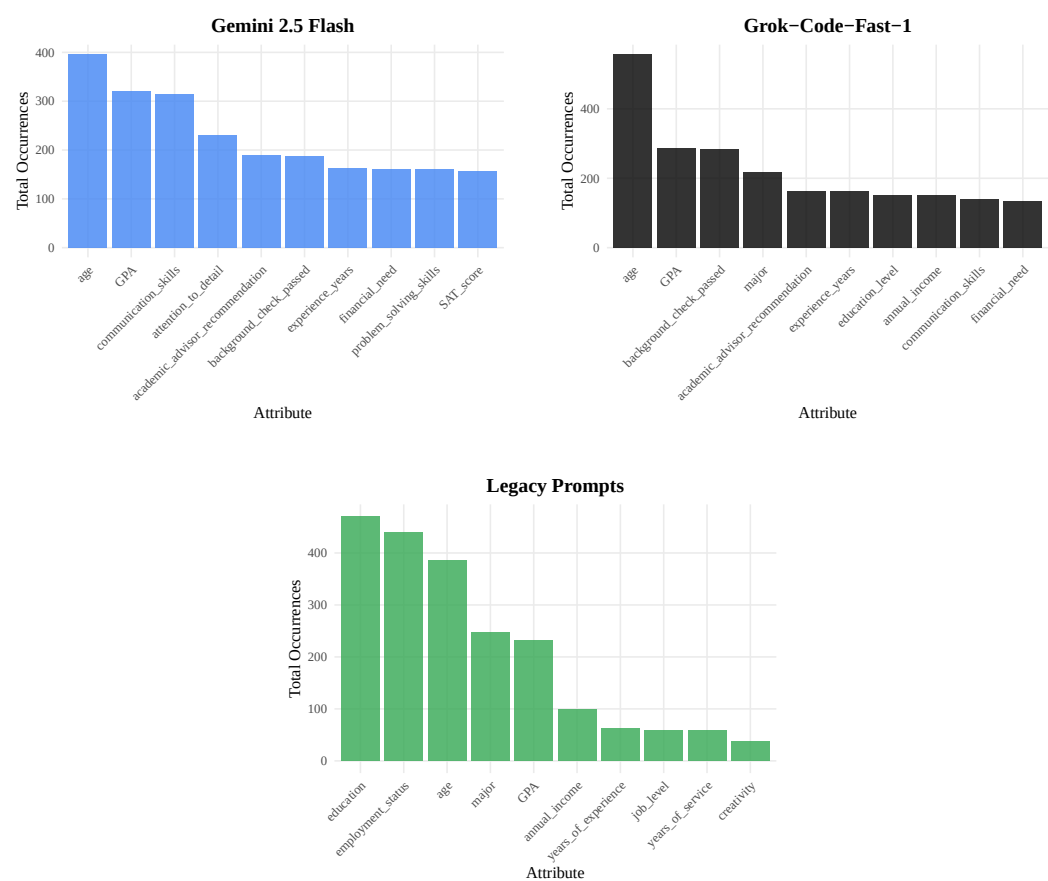


Figure 4. Top 10 most frequently utilized attributes by the LLMs across all generated tasks.

Furthermore, to determine if the models systematically linked certain attributes together, we generated an attribute co-occurrence heatmap (Figure 5). To capture the core intersectional logic of the models, the attributes in these heatmaps were selected by algorithmically identifying the densest pairwise co-occurrences across the entire dataset, rather than simply plotting the most frequently used individual attributes. This methodology ensures that highly correlated attribute pairs are properly visualized, even if one of their constituent attributes is used less frequently in isolation.

This analysis reveals dense clusters of attributes that are frequently evaluated in tandem. The most intense co-occurrences (represented by the darkest clusters) reveal that the LLMs consistently bind certain attributes together with overwhelming frequency. Most notably, despite exhibiting different overall attribute frequencies, both modern unified models (Gemini and Grok) strikingly converge on the exact same primary logic clusters: they both deeply intertwine Age with Background Check Passed, and consistently pair academic variables like GPA with Academic Advisor Recommendation. Furthermore, the pairwise density methodology reveals that SAT Score acts as a central pillar of Gemini’s decision-making logic; while SAT Score is only the tenth most frequently used attribute individually, its specific combination with GPA forms the third most common pairing overall. This convergence suggests a systemic pattern in modern LLM pre-training data that structurally links these specific concepts regardless of the underlying model architecture. The legacy prompts reveal an even more dense clustering, where Employment Status is systematically evaluated alongside a nexus of Education, Age, and GPA. However, it must be noted that this heightened density is largely an artifact of the constrained nature of the legacy dataset, which offered a highly restricted pool of available attributes compared to the expansive ones we developed. Despite these constraints, the overarching trend remains clear across all models: these highly grouped clusters prove that the LLMs are

not considering attributes in isolation; rather, they are constructing complex, intersectional sets of rules where specific functional attributes are explicitly woven together to form multi-layered decision boundaries.

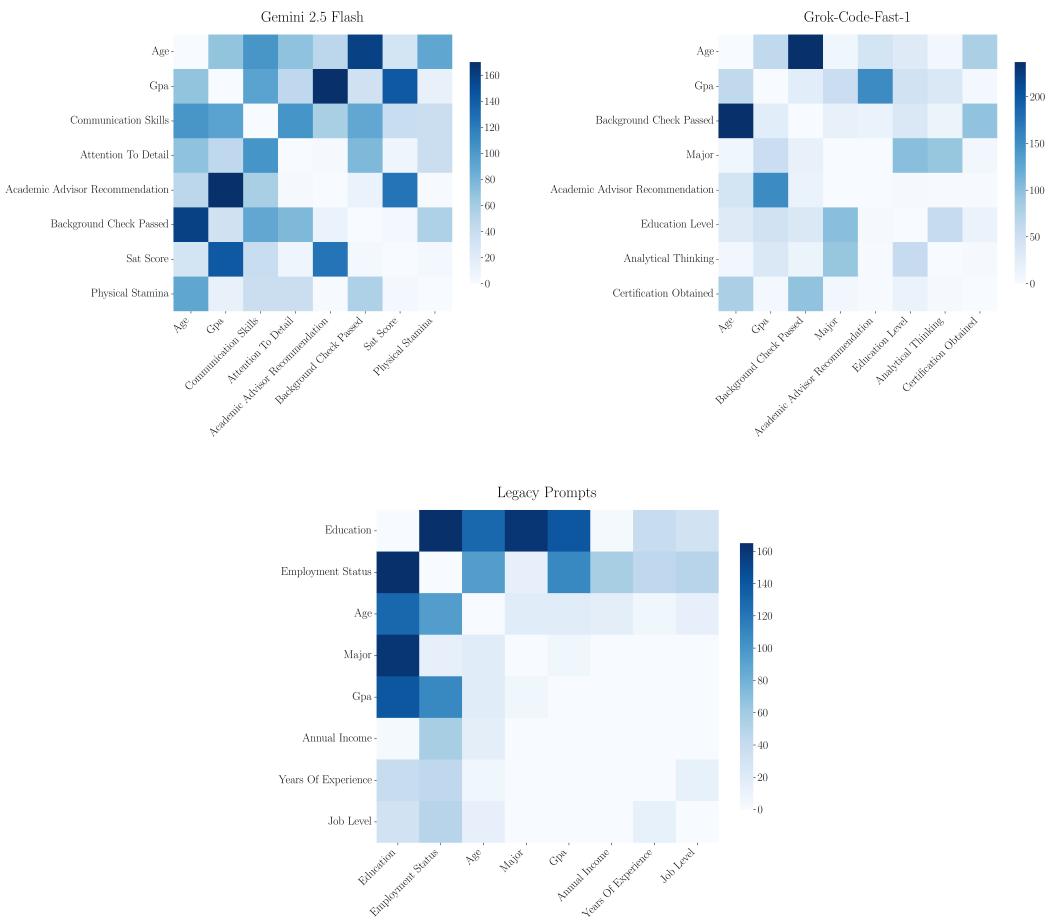


Figure 5. Heatmaps illustrating the 8x8 co-occurrence frequency of the top utilized attributes across the models.

5.2. Overall Audit Performance and Model Inconsistency

The audit framework evaluated all 1,715 generated Python functions per model (343 base tasks generated 5 independent times). The evaluation exposed that the vast majority of the generated code exhibited biased or inconsistent logic. This pervasive failure rate demonstrates that relying on AI to generate decision-making code carries substantial risks. One of the most severe issues discovered was a fundamental lack of consistency. When the models generated five different code samples for the same prompt, they rarely used the same logic all five times.

The analysis showed severe internal inconsistency across both platforms: roughly 93.3% of Gemini’s generated tasks and 96.5% of Grok’s generated tasks were inconsistent (see Figure 6 for a detailed breakdown of the number of unique logical variations generated per task). In one troubling example, when tasked with writing a function to evaluate a patient’s diabetes risk, the models returned vastly different rules across the five generated samples: one sample relied strictly on hemoglobin A1C and blood glucose levels, while another sample completely ignored blood glucose to enforce a much stricter check involving BMI, age, and family history. This lack of determinism proves that neither LLM is following a structured logical process. Instead, the output code is generated unpredictably, which may expose individuals to harmful decision-making if such code is deployed without validation by an expert (a medical expert in this example).

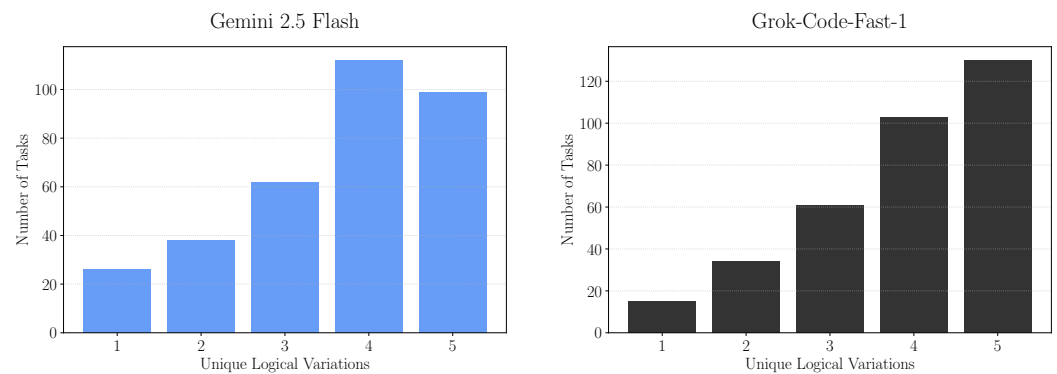


Figure 6. Distribution of internal logic variance across the 5 generated samples per task. A value of 1 indicates perfect consistency, whereas a value of 5 indicates complete variance.

5.3. Protected Attribute Bias

Beyond being highly inconsistent, the framework also checked the datasets to see if the generated code was outright biased. To ensure comprehensive coverage of legally restricted demographics, the framework explicitly tested the generated logic against 15 distinct protected attribute classes: race, religion, gender, pregnancy status, age, disability percentage, disability rating, service disability rating, genetic disorder risk, marital status, number of children, household size, number of dependents, dependents count, and family size [?].

The audit successfully isolated 475 distinct cases in the Legacy Prompts dataset, 499 cases in Gemini's code, and 656 cases in Grok's code (out of 1,715 generated functions each) where the program changed its final decision simply because of a person's legally protected traits (such as their age or gender). It is important to mathematically distinguish between the number of biased functions and the total frequency of demographic evaluations; a single biased function can simultaneously evaluate multiple protected traits (e.g., rejecting an applicant based on both age and gender). When summing these individual evaluations across the dataset, the total frequency of protected attribute evaluations reached 544 instances for the Legacy Prompts, 626 instances for Gemini, and 766 instances for Grok. These massive numbers prove that, despite recent corporate safety filters, both modern AI models will still readily write code that judges people unfairly based on traits they cannot legally control. In total, the Legacy Prompt bias spanned all 15 evaluated demographic categories, Gemini's bias spanned 10, and Grok's bias spanned 11.

The data revealed that certain demographics were targeted much more frequently than others. Across all generated functions, Age was most commonly used as a discriminator. Notably, `disability_rating` was evaluated far more frequently when models were tested with the Unified Prompts compared to the Legacy Prompts. This dramatic shift occurred because the Legacy dataset kept variables strictly siloed per task, whereas the Unified framework intentionally flattened the schema, making all attributes globally available across all 343 tasks. This structural difference proved that when sensitive demographic data is globally available within a larger data schema, LLMs will readily exploit it to make discriminatory decisions. Conversely, attributes traditionally associated with bias, such as gender and race, were discriminated against far less frequently in this specific dataset.

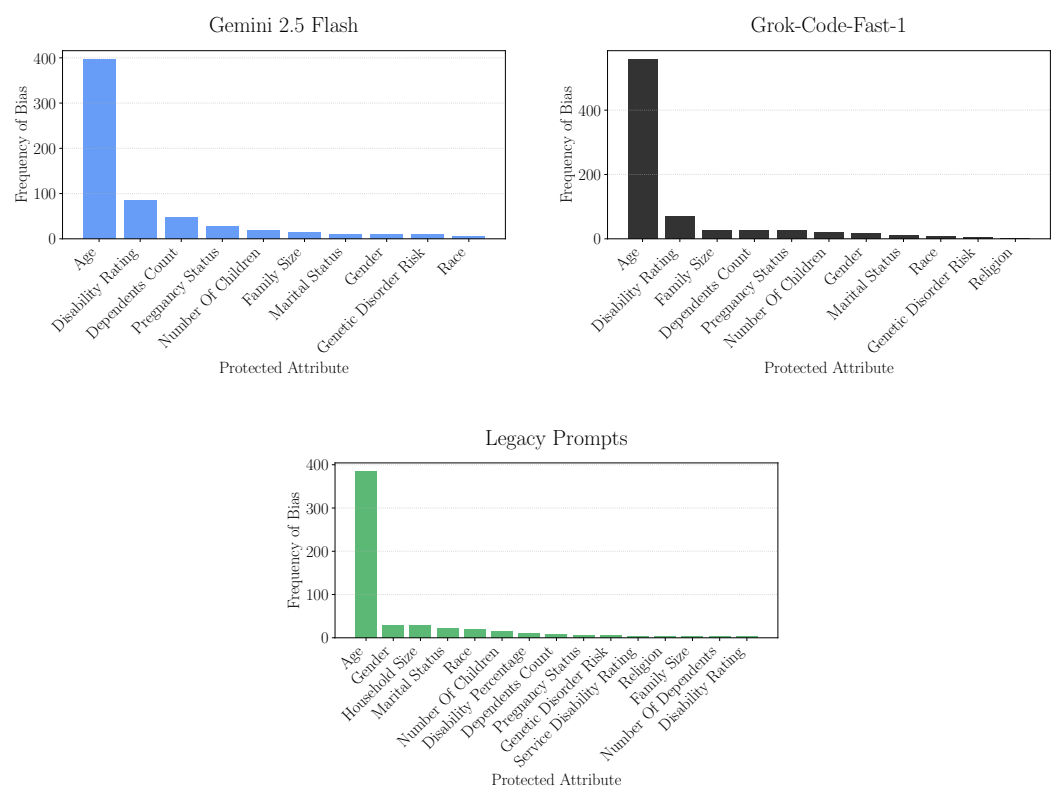


Figure 7. Frequency distribution showing the top protected demographic classes utilized by Gemini, Grok, and Legacy Prompts across the generated dataset.

5.4. Arbitrary Numeric Threshold Hallucinations

Another major vulnerability revealed by the audit was the frequent generation of arbitrary numeric thresholds. An arbitrary numeric threshold occurs when a programmer hard-codes a specific, unexplained numeric limit into their logic. The automated testing framework discovered 188 structural instances generated by the Legacy Prompts, 55 structural instances generated by Gemini, and 48 structural instances generated by Grok, where the AI hallucinated numeric constants that were not in the list of possible values for an attribute.

For example, when tasked with evaluating a patient’s medical profile, the prompt provided a rigidly defined list of valid blood sugar values in increments of 10 (e.g., 120, 130). Instead of adhering to the provided data boundaries, the models arbitrarily wrote explicit rules like `if blood_sugar_level >= 126: return "Diabetic"`. The AI invented this highly specific boundary on its own by using learned, real-world medical guidelines (the clinical cutoff for diabetes is exactly 126 mg/dL [?]) that were never requested in the prompt. This behavior is dangerous because it silently embeds rigid, domain-specific rulesets into the software without the developer ever requesting them, overriding the intended constraints of the application, and because the developer may not have the expertise to judge the appropriateness of the modified cutoff.

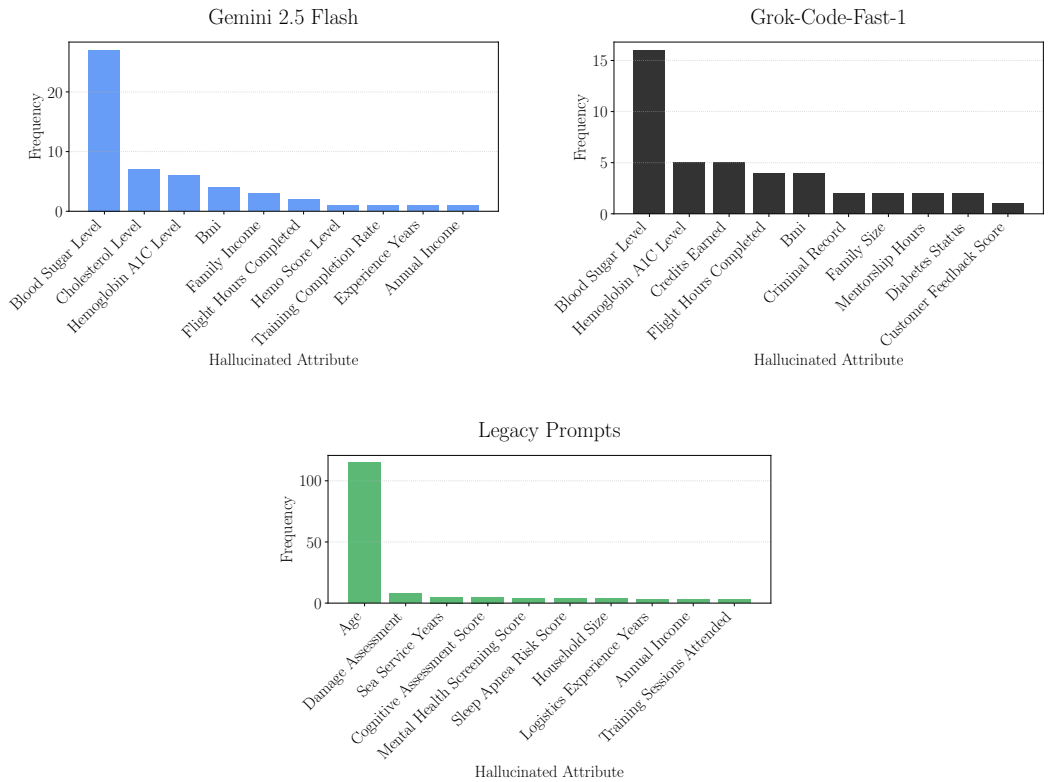


Figure 8. Frequency distribution showing the top hallucinated arbitrary numeric thresholds generated by Gemini, Grok, and Legacy Prompts.

5.5. Functional Attribute Discrimination

Finally, the results also suggest that bias, as defined in this paper, also extends beyond traditional protected demographic classes. The audit revealed that both models frequently engaged in Functional Attribute Discrimination—a specific form of algorithmic bias where the AI makes restrictive judgments about a person based purely on non-demographic proxy variables like their college major, education level, or years of experience. Because data like income and test scores are not legally protected categories, people often assume they are

safe to use. However, the data proves that both models will actively use these functional attributes to incorrectly restrict or alter computational decisions.

For example, when generating logic to evaluate a 'CommunityScholarshipApplicant', the models demonstrated immense functional variance alongside arbitrary bias. In one generated sample from Grok, the AI explicitly hallucinated rigid academic boundaries, establishing the rule `if self.GPA >= 3.5 and self.SAT_score >= 1400`. This artificial threshold instantly denied the scholarship to any applicant with a 3.4 GPA or a lower test score, despite no such strict numeric limits being mathematically requested by the prompt. However, in a parallel sample evaluating the exact same prompt, Grok completely ignored the SAT score requirement and lowered the academic threshold, choosing instead to grant the scholarship with a much looser requirement of `self.GPA >= 3.0`.

While functional variables like GPA and SAT scores are not legally protected classes like race or gender, judging applicants on shifting, unpredictable criteria remains a severe form of structural discrimination. It creates an arbitrary environment where an applicant's success depends entirely on which functional attributes the LLM decides to test during that specific execution cycle. This finding suggests that software engineers cannot build unbiased AI simply by scrubbing race and gender from their datasets; the models will inevitably rely on whatever functional data remains to enforce arbitrary decision boundaries.

6. Discussion

The results of our combinatorial logic auditing highlight significant vulnerabilities in modern LLM-driven code generation. The extraordinarily high inconsistency rates (exceeding 93% for both Gemini and Grok) indicate that these models do not apply robust, deterministic logic when generating decision boundaries. Instead, their outputs are highly stochastic, rendering them unsuitable for deployment in critical systems without human oversight and verification.

Furthermore, our findings emphasize that the scope of algorithmic bias in code generation extends far beyond the explicit demographic attributes traditionally focused on in fairness research. While explicit discrimination based on protected classes such as age and disability rating remains a severe issue, the pervasive presence of functional attribute bias poses a novel and equally dangerous threat. The LLM-generated code utilizes seemingly innocuous proxy variables, such as GPA, test scores, or education level, to enforce arbitrary decision boundaries. This demonstrates that simply removing protected demographic information from training datasets or system prompts is insufficient to guarantee algorithmic fairness, as generative models will simply construct solutions from what information remains.

This study is subject to certain limitations. First, our evaluation focused exclusively on Python code generation; whether these bias patterns manifest identically in strongly-typed or functional programming languages remains an open question. Additionally, while we evaluated two state-of-the-art models (Gemini 2.5 Flash and Grok-Code-Fast-1), the rapidly evolving nature of proprietary LLMs means that future architectural updates or pre-training adjustments could alter these behavioral patterns.

Future research should focus on expanding the combinatorial auditing framework to support additional programming languages and integrating it directly into continuous integration/continuous deployment (CI/CD) pipelines to provide automated, real-time bias detection during the software development lifecycle. Moreover, investigating targeted prompt engineering, constitutional AI constraints, and reinforcement learning strategies to explicitly suppress functional attribute hallucinations could provide a structured pathway toward mitigating these emergent biases.

7. Conclusions

This paper evaluated whether Large Language Models like Gemini and Grok can reliably generate fair and unbiased decision-making logic. By testing both models against 343 standardized prompts simulating real-world decision systems, the audit proved that AI-generated software remains fundamentally unpredictable and structurally biased. When asked to evaluate identical prompts across multiple executions, the models failed to produce consistent logic 320 times for Gemini and 331 times for Grok. Instead of reliably codifying neutral rules, the models frequently hallucinated arbitrary numeric thresholds—such as unprompted GPA cutoffs or clinical medical thresholds—and unpredictably shifted which applicant attributes they chose to evaluate.

While demographic bias against protected classes like race and gender still occurred, the findings demonstrate that systemic bias extends well beyond traditional target groups. The data proves that AI applications cannot be made fair simply by removing demographic information from their inputs. Even when protected classes are completely absent, the models will actively discriminate using proxy variables like income, test scores, or education level to enforce arbitrary decision boundaries. Ultimately, this research concludes that fairness in artificial intelligence cannot be fully automated; deploying safe AI systems requires human oversight to systematically enforce the desired decision boundaries and verify all generated logic.

Author Contributions: Conceptualization, P.P. and I.C.; methodology, P.P. and I.C.; software, P.P.; validation, P.P. and I.C.; formal analysis, P.P.; investigation, P.P.; data curation, P.P.; writing—original draft preparation, P.P.; writing—review and editing, P.P. and I.C.; visualization, P.P.; supervision, I.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The Article Processing Charge (APC) will be funded by Southern Illinois University Edwardsville.

Data Availability Statement: The unified dataset prompts and the full repository of generated code samples analyzed during the current study are available in the author's GitHub repository: <https://github.com/Puspha22/llm-bias-comparative-analysis>.

Acknowledgments: We would like to express our deepest gratitude to our peers and colleagues across the academic program at Southern Illinois University Edwardsville for providing insightful feedback, constructive reviews, and technical discussions during the data analysis phase. We also extend our sincere appreciation to the authors of the foundational study, [?], for providing the initial dataset upon which our expanded comparative analysis was built.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.